



Bancos de Dados Paralelos



Eduardo Ogasawara

eogasawara@cefet-rj.br | <https://eic.cefet-rj.br/~eogasawara>



Paralelismo em Bancos de Dados

Particionamento de Dados

Os dados podem ser distribuídos entre múltiplos discos para permitir operações de entrada e saída paralelas, maximizando a eficiência do sistema.

Operações Paralelas

Operações relacionais como sort, join e agregação podem ser executadas simultaneamente, com cada processador trabalhando em sua própria partição de dados.

Consultas Simultâneas

Diferentes consultas podem ser executadas em paralelo, com o controle de concorrência gerenciando possíveis conflitos entre operações.

As consultas são expressas em linguagem de alto nível como SQL e traduzidas para álgebra relacional, tornando a implementação do paralelismo muito mais fácil e eficiente. Os bancos de dados são, portanto, candidatos naturais para arquiteturas paralelas.

Paralelismo de E/S

Conceito Fundamental

O paralelismo de E/S (entrada/saída) reduz significativamente o tempo necessário para recuperar relações do disco, através do particionamento estratégico dos dados em múltiplos discos físicos.

Particionamento Horizontal

No particionamento horizontal, as tuplas de uma relação são divididas entre diversos discos, de modo que cada tupla individual resida em exatamente um disco. Esta técnica permite acesso simultâneo aos dados, aumentando drasticamente a velocidade de recuperação.

Técnicas de Particionamento

Considerando n discos disponíveis

Rodízio (Round-Robin)

A i^{a} tupla inserida na relação é enviada ao disco calculado por $i \bmod n$. Esta técnica garante distribuição uniforme sem necessidade de análise dos dados.

Particionamento por Hash

Escolha um ou mais atributos como atributos de particionamento e defina uma função de hash h com intervalo de 0 a $n-1$. O resultado de h aplicado ao valor do atributo determina o disco de destino da tupla.

Particionamento por Intervalo

Define-se um vetor de particionamento que mapeia intervalos de valores de atributos para discos específicos, permitindo agrupamento natural dos dados.

Paralelismo por Intervalo

O particionamento por intervalo divide os dados, baseado em ranges específicos de valores de atributos. Por exemplo, em uma base de clientes, podemos particionar por faixas etárias: 18-30 anos no disco 1, 31-50 anos no disco 2, e 51 anos ou mais no disco 3.

Esta técnica oferece vantagens significativas para consultas que envolvem intervalos de valores, pois apenas os discos relevantes precisam ser acessados. O agrupamento natural dos dados permite otimizações importantes no processamento de queries.

Comparação de Técnicas de Particionamento

Para avaliar a eficácia das técnicas de particionamento, devemos considerar três tipos fundamentais de acesso aos dados:

Varredura Completa

Leitura sequencial da relação inteira, processando todas as tuplas do início ao fim.

Consultas Pontuais

Localização de tuplas específicas usando busca associativa, como encontrar registros onde $r.A = 25$.

Consultas de Intervalo

Localização de tuplas cujos valores de atributo estão em um range especificado, por exemplo, $10 \leq r.A < 25$.

Vamos comparar como rodízio, hash e intervalo se comportam em cada cenário.

Análise de Particionamento por Rodízio

Vantagens

- Ideal para varredura sequencial completa da relação
- Distribuição uniforme: todos os discos têm aproximadamente o mesmo número de tuplas
- Balanceamento perfeito de carga entre os discos durante a recuperação
- Simplicidade de implementação

Limitações

As consultas de intervalo são extremamente difíceis de processar com eficiência. Como não há agrupamento natural dos dados, as tuplas relevantes ficam espalhadas por todos os discos do sistema, exigindo acesso a múltiplos discos mesmo para intervalos pequenos.

Análise de Particionamento por Hash

Acesso Sequencial

Excelente para varredura completa. Assumindo uma boa função de hash e atributos de particionamento formando uma chave, as tuplas são distribuídas uniformemente entre os discos, garantindo balanceamento efetivo de carga.

Consultas Pontuais

Desempenho ótimo para buscas sobre o atributo de particionamento. É possível pesquisar um único disco, deixando os outros disponíveis para responder a outras consultas. Índices locais tornam buscas e atualizações mais eficientes.

Consultas de Intervalo

Desempenho ruim devido à ausência de agrupamento. Os valores em um intervalo são distribuídos aleatoriamente entre os discos, exigindo acesso a múltiplos discos mesmo para ranges pequenos.

Análise de Particionamento por Intervalo

O particionamento por intervalo oferece agrupamento natural dos dados baseado nos valores dos atributos de particionamento, proporcionando benefícios únicos para diversos tipos de consultas.

Vantagens Principais

- Excelente para acesso sequencial com boa distribuição
- Ótimo para consultas pontuais: apenas um disco precisa ser acessado
- Superior para consultas de intervalo: poucos discos são necessários
- Discos não utilizados ficam disponíveis para outras queries

Limitação Importante

Quando muitos blocos precisam ser recuperados de poucos discos, o paralelismo potencial no acesso é desperdiçado. Este é um exemplo clássico de distorção de execução, onde o benefício do paralelismo é limitado pela concentração de dados.

Distorção em Particionamento

A distribuição de tuplas entre discos pode ser distorcida, resultando em alguns discos sobrecarregados enquanto outros permanecem subutilizados. Este desequilíbrio compromete seriamente o desempenho paralelo.

Distorção de Valor de Atributo

Ocorre quando determinados valores aparecem com alta frequência nos atributos de particionamento. Todas as tuplas com o mesmo valor acabam na mesma partição, criando um hotspot.

Pode ocorrer tanto com particionamento de intervalo quanto com particionamento de hash, especialmente quando há valores dominantes nos dados.

Distorção de Partição

No particionamento por intervalo, um vetor de particionamento mal escolhido pode atribuir muitas tuplas a algumas partições e poucas a outras.

Menos provável no particionamento de hash, desde que uma boa função de hash seja escolhida, pois a distribuição tende a ser mais uniforme.

Tratamento da Distorção usando Histogramas

Ideia Central

Construir um vetor de particionamento **V** balanceado a partir de um histograma da distribuição dos dados, eliminando a distorção (*skew*) no particionamento por intervalo.

Fórmula Essencial

Cada intervalo do vetor deve conter aproximadamente:

$$\frac{|R|}{n} \text{ tuplas}$$

Parâmetros

- **H** = histograma com **b** buckets
- **f_i** = frequência do bucket *i*
- **n** = número de partições alvo
- **V** = vetor de particionamento resultante

ENTRADA: Relação **R**, número de partições **n**

PRÉ-CONDIÇÃO: atributo de particionamento definido; **n** ≥ 2

PASSO 1 — Construir histograma **H** de **R**:

 amostrar tuplas (varredura, amostragem aleatória

 ou amostragem de blocos)

 contar frequência **f_i** por bucket *i*

PASSO 2 — Calcular alvo por partição:

 alvo ← |**R**| / **n**

PASSO 3 — Percorrer **H** acumulando frequências:

 acum ← 0

 Para cada bucket *i* de **H**:

 acum ← acum + **f_i**

 se acum ≥ alvo:

 adicionar limite superior do bucket *i* a **V**

 acum ← 0

PASSO 4 — Ajustar **V** para exatamente **n-1** pontos de divisão

SAÍDA: Vetor de particionamento **V** = [**v₁**, **v₂**, ..., **v_{n-1}**]

PÓS-CONDIÇÃO: cada partição **P_i** contém ≈ |**R**| / **n** tuplas

Particionamento de Processador Virtual

A distorção no particionamento de intervalo pode ser tratada de forma elegante usando a técnica de particionamento de processador virtual, que oferece uma solução sofisticada para o problema de balanceamento de carga.

Criar Partições Virtuais

Crie uma grande quantidade de partições virtuais, tipicamente de 10 a 20 vezes o número de processadores físicos disponíveis no sistema.

Atribuir Processadores

Cada partição virtual é tratada como um processador virtual. Os processadores virtuais são então atribuídos aos processadores reais em rodízio ou com base no custo estimado de cada partição.

Distribuir Carga

Se qualquer partição for distorcida, a distorção se espalha por diversas partições virtuais, que são distribuídas entre processadores, garantindo trabalho uniforme.

Paralelismo Entre Consultas

O paralelismo entre consultas permite que múltiplas queries e transações sejam executadas simultaneamente, aumentando significativamente o throughput do sistema. Esta é a forma mais direta de paralelismo para implementar.

Arquiteturas Compartilhadas

Em bancos de dados paralelos de memória compartilhada, a implementação é relativamente simples, pois até mesmo sistemas sequenciais já suportam processamento concorrente básico.

Desafios em Arquiteturas Distribuídas

Em arquiteturas de disco compartilhado ou nada compartilhado, a complexidade aumenta: bloqueio e logging precisam ser coordenados via passagem de mensagens, dados em buffers locais podem estar desatualizados, e a coerência de cache deve ser mantida.

Paralelismo Intraconsulta

A execução de uma única consulta em paralelo através de múltiplos processadores e discos é crucial para acelerar consultas de longa duração e melhorar significativamente o tempo de resposta.

Paralelismo Intraoperação

Paraleliza a execução de cada operação individual dentro da consulta. Esta abordagem divide o processamento de uma única operação entre múltiplos processadores.

O paralelismo intraoperação geralmente se expande melhor com o aumento do número de processadores, pois o número de tuplas processadas por cada operação normalmente é muito maior que o número de operações em uma consulta.

Paralelismo Entre Operações

Executa diferentes operações de uma expressão de consulta simultaneamente, aproveitando a independência entre operações sequenciais.

Classificação Paralela

Ordenar grandes relações distribuindo partições entre processadores, onde cada um classifica localmente — sem necessidade de merge global.

Particionamento por intervalo com vetor:

$V = [v_1, v_2, \dots, v_{n-1}]$

- Tuplas com chave $\leq v_1 \rightarrow$ disco 0
- $v_1 < \text{chave} \leq v_2 \rightarrow$ disco 1
- ... e assim sucessivamente

Parâmetros:

- n = número de processadores
- V = vetor de particionamento
- P_i = partição local do processador i

ENTRADA: Relação R , vetor de particionamento V , n processadores

PRÉ-CONDIÇÃO: V possui $n-1$ pontos de divisão balanceados

PASSO 1 — Amostrar R para estimar distribuição dos valores de chave

PASSO 2 — Construir V a partir do histograma amostrado,
garantindo $|P_i| \approx |R| / n$ para todo i

PASSO 3 — Para cada tupla $t \in R$:
determinar disco destino $d \leftarrow$ índice em V
enviar t ao processador P^d

PASSO 4 — Cada processador P_i ordena P_i localmente
(ex.: merge-sort externo)

PASSO 5 — Concatenar $P_0, P_1, \dots, P_{n-1}$ (sem merge):
resultado já está globalmente ordenado

SAÍDA: R ordenada distribuída entre os n processadores

PÓS-CONDIÇÃO: $\forall t \in P_i, \forall s \in P_j$ com $i < j$ $\text{chave}(t) \leq \text{chave}(s)$

Junção Paralela

A operação de junção é uma das mais custosas em bancos de dados. Algoritmos paralelos dividem os pares de tuplas a serem testados entre vários processadores, e cada um calcula sua parte da junção de forma independente.

Junção Particionada (Hash Join Paralelo)

Ideia central: particionar R e S com a mesma função de hash h nos atributos de junção, garantindo que tuplas correspondentes cheguem ao mesmo processador — join local sem comunicação.

Fórmula essencial:

$$\text{destino}(t) = h(t.\text{atributo_junção}) \bmod n$$

Parâmetros:

- **R, S** = relações de entrada
- **h** = função de hash
- **n** = número de processadores
- **R_i, S_i** = partições locais do processador i

ENTRADA: Relações **R** e **S**, função de hash **h**, **n** processadores

PRÉ-CONDIÇÃO: **h** distribui uniformemente os valores do atributo de junção

PASSO 1 — Para cada tupla $r \in R$:

$d \leftarrow h(r.\text{atributo_junção}) \bmod n$

enviar r ao processador $P^d \rightarrow$ forma R^d

PASSO 2 — Para cada tupla $s \in S$:

$d \leftarrow h(s.\text{atributo_junção}) \bmod n$

enviar s ao processador $P^d \rightarrow$ forma S^d

PASSO 3 — Cada processador P_i executa junção **local**:

Para cada $r \in R_i$, para cada $s \in S_i$:

se $r.\text{atributo_junção} = s.\text{atributo_junção}$:

emitir $(r \bowtie s)$

PASSO 4 — Coletar resultados de $P_0, P_1, \dots, P_{n-1}$

SAÍDA: $R \bowtie S$ completo, sem duplicatas

PÓS-CONDIÇÃO: nenhuma comunicação entre processadores no Passo 3

Paralelismo Entre Operadores

O paralelismo entre operadores, também conhecido como paralelismo em pipeline, permite que diferentes operações de uma consulta sejam executadas simultaneamente em um fluxo contínuo de dados.

Nesta abordagem, enquanto um operador processa um conjunto de tuplas, ele passa os resultados intermediários diretamente para o próximo operador, que já pode começar seu processamento. Isso elimina a necessidade de materializar resultados intermediários em disco, economizando tempo de E/S e espaço de armazenamento.

Por exemplo, em uma query com seleção seguida de projeção e então junção, as tuplas podem fluir através desses operadores continuamente, com cada estágio processando diferentes tuplas simultaneamente. Esta técnica é especialmente efetiva quando os operadores têm tempos de processamento similares.

Limitações do Paralelismo Canalizado

Embora útil, o paralelismo em pipeline apresenta limitações importantes que restringem sua aplicabilidade e escalabilidade em sistemas de banco de dados paralelos de grande escala.

Escalabilidade Limitada

Útil com pequena quantidade de processadores, mas não se expande bem além disso. As cadeias de pipeline raramente conseguem tamanho suficiente para utilizar efetivamente dezenas ou centenas de processadores.

Operadores Bloqueantes

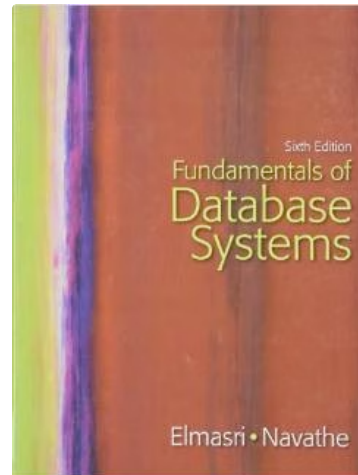
Impossível canalizar operadores que não produzem saída até que todas as entradas tenham sido processadas, como agregação completa (SUM, AVG) e operações de ordenação (ORDER BY).

Distorção de Execução

Pouco ganho é obtido em casos frequentes onde o custo de execução de um operador é muito maior que os outros, criando um gargalo que limita o throughput de todo o pipeline.

Por essas razões, o paralelismo intraoperação geralmente oferece melhores oportunidades de escalabilidade e desempenho em sistemas modernos.

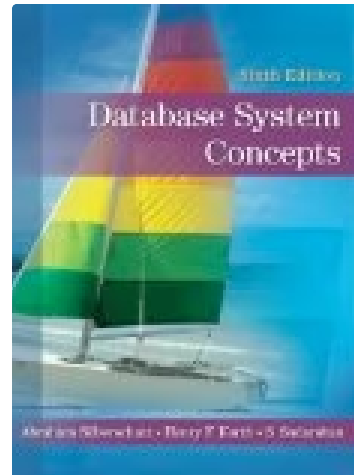
Referências



Elmasri & Navathe

Fundamentals of Database Systems

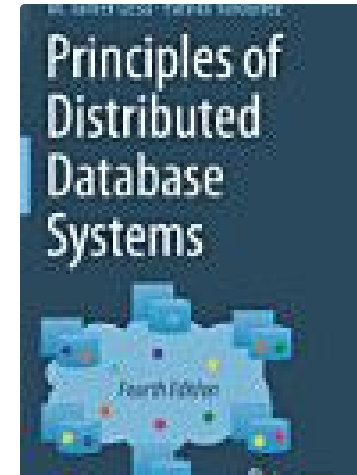
Pearson, 2016



Korth, Sudarshan & Silberschatz

Database System Concepts

McGraw-Hill, 2019



Özsü & Valduriez

Principles of Distributed Database Systems

Springer Nature, 2019